AGENTIC WORLD MODELING FOR AGENT TEST-TIME SCALING WITHOUT ENVIRONMENT REPLICA

Anonymous authors
Paper under double-blind review

ABSTRACT

1 INTRODUCTION

Large language model agents (Yang et al., 2024; Ghafarollahi & Buehler, 2025; Song et al., 2023) have emerged as a powerful paradigm for solving open-ended tasks through iterative reasoning, tool use, and interaction with external environments. However, a single sampled trajectory is often brittle, because early mistakes, incomplete exploration, or unlucky tool calls can irreversibly derail the agent. Test-time scaling (Snell et al., 2025; Muennighoff et al., 2025) offers a promising way to improve reliability by allocating additional computation during inference. In conventional reasoning domains, test-time scaling is often instantiated by sampling multiple responses simultaneously. For LLM agents, however, test-time scaling is not merely a matter of generating text, but also executing real actions in parallel with external environments, thus necessitating multiple environment copies.

In realistic deployments, however, modern agents may operate over enterprise workspaces, cloud services, multi-user collaboration platforms, etc. Forking independent copies of such environments for every sampled trajectory is often infeasible due to storage, privacy, or access-control constraints. Moreover, speculative trajectories may execute persistent or irreversible actions (Xu et al., 2025; Koh et al., 2025), such as deleting records or modifying shared resources. Executing multiple sampled trajectories against the same live environment can therefore cause cross-trajectory interference. Consequently, naive parallel sampling against a single real environment is generally unsuitable as a foundation for agentic test-time scaling.

An alternative is to equip the agent with an internal transition model that predicts the next environment state after each hypothetical action, e.g., by using the LLM itself as a world model conditioned on the current state and action (Hao et al., 2023; Wang et al., 2024a). However, faithful prediction requires a transition-sufficient representation of the environment, which is often infeasible in realistic settings where relevant state spans large, interdependent artifacts (e.g., files, messages, databases) and may exceed practical context limits (Tang et al., 2026). As a result, agent systems typically approximate state via summarization or heuristic retrieval (Chae et al., 2025a; Mei et al., 2026), but both approaches are lossy: summaries may omit or distort facts, and retrieval may miss dependencies that become relevant only under hypothetical actions, leading to hallucinated details.

To address this limitation, we reformulate next-state prediction from a single-turn generation problem into an agentic process that interacts with the real environment over multiple turns. This approach avoids compressing the entire environment into a single prompt. Instead, the simulator can adaptively identify missing information, retrieve it through safe read-only queries, and ground its prediction of the resulting state in the actual environment. The design retains the main benefit of test-time scaling—the ability to explore multiple candidate trajectories in parallel—while preventing speculative rollouts from mutating or interfering with the shared environment. It also recasts world modeling as an interactive reasoning problem: the simulator determines what evidence is needed, acquires that evidence through non-destructive interactions, and uses it to produce predictions that are more consistent with the environment's true state.

2 Related Works

2.1 Test-time Scaling of Agent.

Test-time scaling (TTS) (Snell et al., 2025; Muennighoff et al., 2025) has become an important paradigm for improving the reliability of large language models by allocating additional computation at inference time. While early TTS methods were primarily developed for static reasoning tasks, recent work (Koh et al., 2024; Zhou et al., 2023; Yang et al., 2026; Zhu et al., 2025; Lee et al., 2026a; Kim et al., 2026) has extended TTS to agents by generating multiple candidate trajectories and selecting or aggregating their predicted outcomes (Lee et al., 2026b; Hamid et al., 2026). While some methods further incorporate reflection, verification and sequential revision to refine candidate trajectories (Shinn et al., 2023; Gou et al., 2024; Wan et al., 2026). Collectively, these studies demonstrate that inference-time computation can improve not only an agent's internal reasoning, but also its ability to explore, plan, and recover from mistakes. Despite these advances, such approaches still require the environment to be cloneable or resettable. However, this assumption is difficult to satisfy in modern enterprise settings. Our work targets this problem: instead of assuming multiple writable replicas of the environment, we seek to enable TTS through grounded prediction over a single real environment while avoiding unsafe parallel mutation.

2.2 World Modeling.

World models provide agents with predictive representations of environment dynamics, enabling planning over imagined futures and learning from simulated experience. They have been widely studied in model-based reinforcement learning, video prediction, and robotics, where models forecast future states to support control (Hafner et al., 2023; Bruce et al., 2024; Zhou et al., 2025; Assran et al., 2025). More recently, world modeling for LLM agents has expanded to include planning systems that simulate action outcomes before execution (Gu et al., 2025; Chae et al., 2025b), fully executable synthetic environments for agent training (Wang et al., 2026), and language models trained for next-state prediction across diverse interactive domains (Zuo et al., 2026). At inference time, iterative simulation has also been used to evaluate candidate trajectories before committing to real actions (Zeng et al., 2026). Our approach differs in two ways. First, we ground simulation in the real environment via safe read-only access rather than synthetic or resettable replicas. Second, we treat next-state prediction as an interactive inference process: the simulator can actively gather non-mutating evidence instead of relying on a static context. This turns world modeling from passive prediction into evidence-driven simulation while preserving test-time planning benefits.

3 Grounded Environment Simulation (GES)

3.1 Preliminaries & Notations

We model the environment as a partially observable Markov decision process (POMDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \Omega, \gamma)$, where $s_t \in \mathcal{S}$ is the latent environment state, $a_t \in \mathcal{A}$ is an action, and $o_t \in \mathcal{O}$ is the observation returned to the agent. At step t, the agent receives an observation history $h_t = (o_0, a_0, o_1, \ldots, o_t)$ and selects an action under a policy $a_t \sim \pi_\theta(\cdot \mid h_t)$. The latent state then evolves according to the transition dynamics $s_{t+1} \sim \mathcal{T}(\cdot \mid s_t, a_t)$, and emits an observation according to $o_{t+1} \sim \Omega(\cdot \mid s_{t+1})$. For notational convenience, we marginalize over the latent state and fold the transition and observation models into a single *observation-transition model* that acts directly on histories, $o_{t+1} \sim \mathcal{P}(\cdot \mid h_t, a_t)$.

Many test-time scaling methods improve decision quality by exploring multiple candidate futures before committing an action to the real environment. In practice, most existing approaches realize this exploration by rolling out candidate trajectories directly in the true environment $\mathcal{P}$. However, executing such hypothetical trajectories in parallel is often unsafe, costly, or simply impractical in real agentic settings, since actions may be irreversible or have external side effects. A natural alternative is therefore to replace the true environment dynamics $\mathcal{P}$ with a world simulator $\mathcal{P}_\theta$. Given a hypothetical trajectory prefix h_t, the simulator generates future observations according to $o_{t+1} \sim \mathcal{P}_\theta(\cdot \mid h_t, a_t)$, thereby producing simulated trajectories τ. These trajectories are then scored, aggregated, or selected by a verifier, reward model, search procedure, or other test-time scaling mechanism before any action is committed to the real environment.

3.2 NEXT-STATE PREDICTION AS A MULTI-TURN AGENT TASK

Prior work typically instantiates the world simulator as a direct observation predictor, prompting an LLM π_θ to generate the next observation from the current trajectory history and candidate action, $\hat{o}t + 1 \sim \pi\theta(\cdot \mid h_t, a_t)$. This treats environment simulation as a single-shot generative problem. In realistic environments, however, the latent state s_t is far too large to be represented in context, owing to both its scale and privacy constraints, so any static prompt exposes only a partial view of the environment. Our key idea is to transform simulation from a purely generative problem into an evidence-seeking agentic task.

From single-shot generation to evidence-seeking. The premise of our formulation is that reading the environment is both safe and parallelizable: inspecting state does not change it, so evidence can be gathered concurrently and without side effects. We exploit this by granting the simulator access to a set of safe read-only tools $\mathcal{R}$, such as file inspection, metadata retrieval, log queries, database lookups, and read-only API calls. Through these tools, it queries the true environment state before committing to a prediction. Rather than predicting $\hat{o}_{t+1}$ from a static prompt, the simulator interleaves such reads with reasoning and emits its prediction only once it judges the gathered evidence sufficient. Consequently, next-observation prediction is grounded in real environment state rather than in parametric knowledge or partial context, reducing the risk of hallucinated or inconsistent transitions.

Formally, let $\xi_{<i} = (q_1, e_1, \ldots, q_{i-1}, e_{i-1})$ denote the evidence gathered before step i. At each step the simulator acts under a policy whose action space is the union of read-only queries and observation predictions: it either issues another query $q_i \sim \pi_\theta(\cdot \mid h_t, a_t, \xi_{<i})$, or, once it judges the accumulated evidence sufficient, directly predicts the next observation. A query $q_i = (R_i, \kappa_i)$ pairs a read-only tool $R_i \in \mathcal{R}$ with a lookup specification κ_i identifying the information to retrieve (e.g., a file path, database key, metadata field, log filter, API endpoint, or search expression), and executing it against the current state yields a deterministic evidence item $e_i = R_i(\kappa_i; s_t)$. To bound inference cost, we cap the number of reads at a budget M: once M reads have been issued, the simulator must predict at the next step. The simulator therefore adaptively produces an evidence trajectory $\xi_t = (q_1, e_1, \ldots, q_m, e_m)$ whose length $0 \le m \le M$ is itself chosen by the policy, and then predicts the next observation conditioned on it, $o_{t+1} \sim \pi_\theta(\cdot \mid h_t, a_t, \xi_t)$.

An equivalent observation-transition model. Equivalently, GES defines an interactive observation-transition model:

$$\mathcal{P}_\theta^{\mathrm{GES}}(o_{t+1} \mid h_t, a_t) = \sum_{m=0}^{M} \sum_{|\xi_t|=m} \pi_\theta(\xi_t \mid h_t, a_t)\, \pi_\theta(o_{t+1} \mid h_t, a_t, \xi_t), \tag{1}$$

$$\pi_\theta(\xi_t \mid h_t, a_t) = \prod_{i=1}^{m} \pi_\theta(q_i \mid h_t, a_t, \xi_{<i})\, p(e_i \mid q_i, s_t), \tag{2}$$

where $p(e_i \mid q_i, s_t) = 1$ in our setting, since each read-only tool call returns a deterministic result. Here the terminal factor $\pi_\theta(o_{t+1} \mid h_t, a_t, \xi_t)$ implicitly carries the decision to stop reading: because predicting an observation and issuing a further query compete for probability mass within the same action space, the mass placed on prediction is exactly the probability of terminating with o_{t+1} after evidence ξ_t.

This formulation naturally generalizes direct observation prediction. The single-shot LLM simulator is the degenerate case $m = 0$, in which the simulator predicts immediately without issuing any query: the product over queries is empty (hence equal to one), the evidence trajectory reduces to $\xi_t = \varnothing$, and

$$\mathcal{P}_\theta(o_{t+1} \mid h_t, a_t) = \pi_\theta(o_{t+1} \mid h_t, a_t, \varnothing). \tag{3}$$

At the other extreme, $m = M$ corresponds to exhausting the read budget before predicting. Between these limits, GES treats simulation as an adaptive information-acquisition process: the simulator actively determines what information a hypothetical transition requires, retrieves it through safe reads, and decides when the accumulated evidence is adequate before conditioning its prediction on it.

3.3 Practical Implementation in Test-Time Scaling

We now describe how GES is instantiated within a test-time scaling loop. Recall that each candidate action is evaluated by rolling out a simulated trajectory, and each step of that trajectory requires a next-observation prediction that, under GES, is itself a multi-turn evidence-gathering episode. This nesting — trajectories over predictions, and predictions over read-only interactions — raises two practical concerns: keeping the context manageable, and exploiting the additional test-time compute effectively [that new axis for test-tine scaling seems far-fetched from exploiting compute effectively]. We address them below and specify the concrete tool set $\mathcal{R}$ used in our experiments.

Subagent for context management. A single simulated trajectory contains many next-observation predictions, and under GES, each prediction unfolds as its own multi-turn interaction. Retaining every intermediate query and evidence item in a single context would cause the prompt to grow rapidly with trajectory length, quickly exhausting the context window and inflating inference cost. To prevent this, we delegate each next-state prediction to a dedicated subagent. Given the transition query (h_t, a_t), the subagent runs its own evidence-gathering loop over $\mathcal{R}$, conditions on the resulting evidence ξ_t, and returns only the predicted observation o_{t+1} to the parent trajectory; the intermediate evidence trajectory is discarded once the prediction is emitted. The parent rollout therefore accumulates only the predicted observations $(o_1, \ldots, o_{t+1})$, keeping its context at the scale of an ordinary trajectory while the read-heavy interaction is confined to the transient scope of the subagent. Concretely, a subagent is parameterized by the same policy π_θ, the transition query (h_t, a_t) as its input, the read-only tool set $\mathcal{R}$, and the read budget M; it exposes a single output, the predicted next observation.

Environment simulation as a new scaling axis. [the beginning sentence is not suspenceful enough. maybe begin with something more severe?] A natural question is whether simulation quality, like decision quality, improves as we spend more test-time compute on it. We find that it does: the fidelity of the simulator defines a new, complementary scaling axis. Concretely, for each transition we draw K independent evidence-gathering rollouts from the GES policy, yielding candidate predictions $\{o_{t+1}^{(1)}, \ldots, o_{t+1}^{(K)}\}$, and aggregate them with an LLM-as-a-verifier that scores the candidates against the gathered evidence and selects (or synthesizes) the most consistent next observation. Because different rollouts explore different read paths, they surface complementary evidence and reduce the chance that a single hallucinated read corrupts the prediction. Empirically, prediction accuracy increases monotonically with K [report trend, e.g., "improving from $X\%$ at K=1 to $Y\%$ at K=8"], and this gain is orthogonal to trajectory-level scaling: allocating compute to more accurate per-step simulation improves downstream decision quality even at a fixed number of candidate trajectories

Tool set $\mathcal{R}$. Our default read-only tool set comprises three primitives: READ, which returns the contents of a specified data item (e.g., a file or record); LIST, which enumerates the entities related to the current item (e.g., the fields, keys, or neighbors relevant to a term); and TRACE, which follows and returns all logs or dependency chains connected to a given data item or file. Together these cover the common access patterns of point lookup, neighborhood expansion, and provenance tracing, while remaining strictly non-mutating. The set is deliberately extensible: additional read-only tools can be registered for a particular scenario or backend — for instance, schema introspection for relational databases, endpoint queries for APIs, or full-text search for document stores — without any change to the GES formulation, since every tool enters uniformly as an element $R_i \in \mathcal{R}$.

3.4 Theoretical Advantages

Our agentic formulation and a static-summarization baseline face an identical constraint: only a bounded amount of information about the latent state s_t can be placed in the predictor's context, a budget we denote B (in tokens or bits). They differ only in when they decide what to keep. A static summarizer compresses the history into $Z = c(h_t)$ before the candidate action is known, so Z is independent of a_t; GES instead retrieves evidence $\xi_t = c(h_t, a_t)$ after observing a_t. The following result shows that, under a bottleneck, this ordering alone is decisive. Suppose predicting o_{t+1} requires a task-relevant fact $W^\star$ that depends on the realized action, and the prediction is correct only if $W^\star$ is present in the predictor's context of budget B. Let $p = \Pr[W^\star \in Z]$ be the probability that an action-agnostic summary contains the needed fact, and $q = \Pr[W^\star \in \xi_t]$ the corresponding probability for action-conditioned retrieval.

Proposition 3.1 (informal; see Appendix **??**). *Action-conditioned retrieval never does worse, $q \geq p$. Moreover, if distinct actions require distinct facts drawn from N candidates, each of size β, then any action-agnostic summary is limited to $p \leq \frac{\lfloor B/\beta \rfloor}{N}$, whereas action-conditioned retrieval attains $q = 1$ as soon as a single fact fits the budget ($\beta \leq B$). Consequently the prediction-error gap grows to 1 as the environment ($N\beta$) outgrows the budget B.*

The intuition is that committing to a summary before the action is revealed forces the budget to hedge across all N actions the agent might take, so each contingency receives only a $\lfloor B/\beta \rfloor / N$ share; conditioning on a_t removes this hedge and spends the entire budget on the one fact that the transition actually needs. In Appendix **??** we make this exact with a matching information-theoretic bound: for a uniform action distribution, every action-agnostic summarizer with an B-bit context incurs average error at least $1 - \frac{1}{\beta} - \frac{B}{N\beta}$ (via Fano's inequality), while action-conditioned retrieval achieves zero error within the same budget. This separation is the formal counterpart of our design principle: query the environment for what a hypothetical action requires, rather than guessing in advance what any action might require.

4 Experiments

4.1 Experimental Settings

We evaluate GES on three agentic benchmarks spanning distinct interaction modalities, and compare against both training-based and prompting-based world-model baselines under a unified protocol.

Benchmarks. We consider three complementary environments. **τ-bench** (Yao et al., 2024) evaluates tool–agent–user interaction in two customer-service domains (retail and airline): the agent must converse with an LLM-simulated user, follow domain-specific policy documents, and issue API tool calls. **AppWorld** (Trivedi et al., 2024) provides a high-fidelity simulated ecosystem of nine everyday applications exposed through 457 APIs, with 750 tasks grounded in the digital activities of roughly 100 fictitious users; it requires agents to iteratively generate and execute code, coordinate actions across applications, and complete complex workflows without causing unintended state changes. **OfficeBench** (Wang et al., 2024b) evaluates office automation in an integrated environment comprising applications such as a word processor, spreadsheet editor, and email client. Its 300 tasks span one to three applications and require agents to plan over long horizons, switch between applications at appropriate points, and manipulate application-specific content.

Baselines. We compare GES against two representative world-model paradigms. **SIMIA** (Li et al., 2025) (in-context simulation) is a purely generative baseline that verbalizes the environment into the prompt and predicts the next observation in a single shot. This is the standard direct-observation-predictor setting and corresponds to the $m{=}0$ case of GES. **WMT** (Zuo et al., 2026) (world model training) is a world model that acquires environment dynamics through a dedicated trained via the three-stage continued-pretraining, SFT, RL recipe of Zuo et al. (2026).

Implementation details. Unless otherwise noted, we use a sampling temperature of 1.0. We instantiate the simulator with three models spanning the closed/open and reasoning/non-reasoning spectrum: **GPT-4.1**, a closed-source non-reasoning (non-thinking) model; **GPT-5.4**, OpenAI's closed-source frontier reasoning ("Thinking") model released in March 2026, supporting up to a 1M-token context and native tool/computer use; and **Qwen-AgentWorld-35B-A3B**, an open-weight Mixture-of-Experts model (35B total / 3B active parameters, 256K context) that serves both as the Native-WM baseline and, with our scaffold, as a GES backbone. For each model we cap the number of read-only interaction steps at the budget 6 and report the aggregation setting used for test-time scaling. [Add hardware/serving note if needed, e.g., the open model is served in FP8 on a single A100]

4.2 Results of Main Experiments.

Methods	BoN	τ-bench		AppWorld			OfficeBench		
		Airline	Retail	Level1	Level2	Level3	L1	L2	L3
GPT-4.1									
SIMIA (Li et al., 2025)	@1	44.00	5.26	25.58					
	@4	44.00	5.26	35.66					
	@16	46.00	5.26	39.53					
GES	@1	36.00	42.98	62.79					
	@4	56.00	54.39	71.32					
	@16	68.00	70.18	73.64					
GPT-5.4									
SIMIA (Li et al., 2025)	@1	44.00	5.26	20.93	7.07	1.55	15.05	9.47	1.79
	@4	46.00	5.26	25.58	9.09	1.55	17.20	12.63	2.68
	@16	52.00	6.14	31.01	11.11	1.94	27.96	18.95	4.46
GES	@1	60.00	49.12	82.17	52.02	37.60			
	@4	72.00	71.93	85.27	58.59	47.29			
	@16	82.00	77.19	87.60	65.66	53.88			
QWEN3.5-35B-A3B									
SIMIA (Li et al., 2025)	@1								
	@4								
	@16								
WMT (Zuo et al., 2026)	@1								
	@4								
	@16								
GES	@1								
	@4								
	@16								

Table 1: Caption

Methods	BoN	τ-bench		AppWorld			OfficeBench		
		Airline	Retail	Level1	Level2	Level3	L1	L2	L3
GPT-5.4									
SIMIA (Li et al., 2025)	@1	56.00	52.63	88.37	55.05	51.16	59.14	71.58	23.32
	@4	66.00	69.30	90.70	57.58	58.91	64.51	70.53	27.68
	@16	86.00	75.44	92.25	58.59	57.75	67.74	73.68	33.93
GES	@1	56.00	56.14	85.27	67.17	63.18	63.44	67.37	26.79
	@4	84.00	86.84	93.02	73.23	63.57	64.52	75.79	29.46
	@16	86.00	91.23	94.57	93.94	93.02	66.67	78.95	34.82
GPT-5.4									
SIMIA (Li et al., 2025)	@1	44.00	5.26	20.93	7.07	1.55	9.47	1.79	
	@4	46.00	5.26	25.58	9.09	1.55	12.63	2.68	
	@16	52.00	6.14	31.01	11.11	1.94	18.95	4.46	
GES	@1	60.00	49.12	82.17	52.02	37.60			
	@4	72.00	71.93	85.27	58.59	47.29			
	@16	82.00	77.19	87.60	65.66	53.88			
QWEN3.5-35B-A3B									
SIMIA (Li et al., 2025)	@1								
	@4								
	@16								
WMT (Zuo et al., 2026)	@1								
	@4								
	@16								
GES	@1								
	@4								
	@16								

Table 2: Caption

4.3 ANALYZE.

Environment simulation as a new scaling axis.

Test-time scaling with aggregation.

Comparison to the ceiling performance.

Case analyses.

5 CONCLUSION

ACKNOWLEDGEMENT

REFERENCES

Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first international conference on machine learning*, 2024.

Hyungjoo Chae, Namyoung Kim, Kai Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *International Conference on Learning Representations*, volume 2025, pp. 63707–63738, 2025a.

Hyungjoo Chae, Namyoung Kim, Kai Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *International Conference on Learning Representations*, volume 2025, pp. 63707–63738, 2025b.

Alireza Ghafarollahi and Markus J Buehler. Sciagents: automating scientific discovery through bioinspired multi-agent intelligent graph reasoning. *Advanced Materials*, 37(22):2413523, 2025.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, et al. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pp. 57734–57811, 2024.

Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, Huan Sun, and Yu Su. Is your llm secretly a world model of the internet? model-based planning for web agents. *Transactions on Machine Learning Research*, 2025.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

Jubayer Ibn Hamid, Ifdita Hasan Orney, Michael Y Li, Omar Shaikh, Yoonho Lee, Dorsa Sadigh, Chelsea Finn, and Noah Goodman. Spiral: Learning to search and aggregate. *arXiv preprint arXiv:2606.23595*, 2026.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 8154–8173, 2023.

Joongwon Kim, Wannan Yang, Kelvin Niu, Hongming Zhang, Yun Zhu, Eryk Helenowski, Ruan Silva, Zhengxing Chen, Srinivasan Iyer, Manzil Zaheer, et al. Scaling test-time compute for agentic coding. *arXiv preprint arXiv:2604.16529*, 2026.

Jing Yu Koh, Stephen McAleer, Daniel Fried, and Ruslan Salakhutdinov. Tree search for language model agents. *arXiv preprint arXiv:2407.01476*, 2024.

Jing Yu Koh, Stephen Marcus McAleer, Daniel Fried, and Ruslan Salakhutdinov. Tree search for language model agents. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856.

Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. Agentic test-time scaling for webagents. *arXiv preprint arXiv:2602.12276*, 2026a.

Yoonsang Lee, Howard Yen, Xi Ye, and Danqi Chen. Agentic aggregation for parallel scaling of long-horizon agentic tasks. *arXiv preprint arXiv:2604.11753*, 2026b.

Yuetai Li, Huseyin A Inan, Xiang Yue, Wei-Ning Chen, Lukas Wutschitz, Janardhan Kulkarni, Radha Poovendran, Robert Sim, and Saravan Rajmohan. Simulating environments with reasoning models for agent training. *arXiv preprint arXiv:2511.01824*, 2025.

Kai Mei, Jiang Guo, Shuaichen Chang, Mingwen Dong, Dongkyu Lee, Xing Niu, and Jiarong Jiang. R-wom: Retrieval-augmented world model for computer-use agents. In *International Conference on Learning Representations*, volume 2026, pp. 58831–58855, 2026.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 20286–20332, 2025.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations*, volume 2025, pp. 10131–10165, 2025.

Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 2998–3009, 2023.

Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Yukai Wu, Weizheng Wang, Hongzhang Huang, Wei Zhou, Jun Zhou, Jiachen Song, et al. Workspace-bench 1.0: Benchmarking ai agents on workspace tasks with large-scale file dependencies. *arXiv preprint arXiv:2605.03596*, 2026.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, 2024.

Yuxuan Wan, Tianqing Fang, Yintong Huo, Wenxuan Wang, Haitao Mi, Dong Yu, Michael R Lyu, et al. Inference-time scaling of verification: Self-evolving deep research agents via test-time rubric-guided verification. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 24822–24835, 2026.

Ruoyao Wang, Graham Todd, Ziang Xiao, Xingdi Yuan, Marc-Alexandre Côté, Peter Clark, and Peter Jansen. Can language models serve as text-based world simulators? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1–17, 2024a.

Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. Officebench: Benchmarking language agents across multiple applications for office automation. *arXiv preprint arXiv:2407.19056*, 2024b.

Jiakai Xu, Tianle Zhou, Eugene Wu, and Kostis Kaffes. Toward systems foundations for agentic exploration. *arXiv preprint arXiv:2510.05556*, 2025.

John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

Yan Yang, Dongxu Li, Yutong Dai, Yuhao Yang, Ziyang Luo, Zirui Zhao, Zhiyuan Hu, Junzhe Huang, Amrita Saha, Zeyuan Chen, et al. Gta1: Gui test-time scaling agent. In *International Conference on Learning Representations*, volume 2026, pp. 48295–48323, 2026.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Xingshan Zeng, Lingzhi Wang, Weiwen Liu, Liangyou Li, Yasheng Wang, Lifeng Shang, Xin Jiang, and Qun Liu. Artis: Agentic risk-aware test-time scaling via iterative simulation. *arXiv preprint arXiv:2602.01709*, 2026.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.

Gaoyue Zhou, Hengkai Pan, Yann Lecun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 79115–79135. PMLR, 13–19 Jul 2025.

King Zhu, Hanhao Li, Siwei Wu, Tianshun Xing, Dehua Ma, Xiangru Tang, Minghao Liu, Jian Yang, Jiaheng Liu, Yuchen Eleanor Jiang, et al. Scaling test-time compute for llm agents. *arXiv preprint arXiv:2506.12928*, 2025.

Yuxin Zuo, Zikai Xiao, Li Sheng, Fei Huang, Jianhong Tu, Yuxuan Liu, Tianyi Tang, Xiaomeng Hu, Yang Su, Qingfeng Lan, et al. Qwen-agentworld: Language world models for general agents. *arXiv preprint arXiv:2606.24597*, 2026.

A Appendix

You may include other additional sections here.